awk

awk is a scripting language which is used for processing or analyzing text files.

awk is used for grouping of data based on either a column or field, or on a set of columns.

It derives its name from the first letter of the last name of its three authors namely Alfred V. Aho, Peter J.Weinberger and Brian W. Kernighan.

awk can be used for reporting data in a useful manner. It searches one or more files to see if they contain lines that match specified patterns and then perform associated actions. awk is an advanced filter.

Simple awk Filtering

Syntax of awk:

~$ awk 'pattern {action}' input-file
Let's take a input file with the following data
~$cat awk_file
Name,Marks,Max_Marks
Peter,200,1000
Sam,500,1000
Greg,1000
Abharam,800,1000
Henry,600,1000
Peter,400,1000

Example: Default behavior of awk
Print all the lines from a file.

By default, awk prints all lines of a file, so to print every line of above created file , use below command:
~$ awk '{print}' awk_file

Name,Marks,Max_Marks
Peter,200,1000
Sam,500,1000
Greg,1000
Abharam,800,1000
Henry,600,1000
Peter,400,1000

Example 2: Print only specific field

Print 2nd & 3rd fileds
~$ awk -F”,” {print $2,$3;}' awk_file

Example: Pattern Matching
Print the lines which matches the pattern (lines which contains the word “Henry" or "Peter”)
~$ awk '/Henry|Peter/' awk_file
Peter,200,1000
Henry,600,1000
Peter,400,1000

Initialization and Final Action

BEGIN and END blocks are helpfull in displaying information before and after executing actual awk script.

BEGIN block is evaluated before awk starts processing the actual awk script; it’s an excellent place to initialize the FS (field separator) variable, print a heading, or initialize other global variables.

BEGIN block Usages:

- Declaring variables.
- Initialization variables for doing increment/decrements operations in main AWK code.
- Printing Headings/info before actual AWK code output.

END block is evaluated after all the lines in the awk script have been processed. Typically, the END block is used to perform final calculations or print summaries that should appear at the end of the output stream.

END block Usages:

- Printing final results, after doing operations in main AWK block.
- Printing Completion/info after actual AWK code output.

awk tool is mainly used for reporting some data in useful manner. Without these BEGIN and END
blocks the output will be meaningless.
Consider db.txt which contains below data:
Jones 2143 78 84 77
Gondrol 2321 56 58 45
RinRao 2122234 38 37

awk BEGIN block

This is a block of code which is executed before executing actual awk script.
BEGIN block Syntax

awk 'BEGIN{awk initializing code}{actual AWK code}' filename.txt

Example: Print a meaning full info before actual AWK output.

~$ awk 'BEGIN{print "##########\nThis is the output of filtered
data\n##########"}{print $0}' db.txt

Output:

##########
This is the output of filtered data
##########
Jones 2143 78 84 77
Gondrol 2321 56 58 45
RinRao 2122234 38 37
Edwin 253734 87 97 95
Dayan 24155 30 47

awk END block

This is the block which is executed after executing all the awk code.
Example:

Print some meaning full info after processing awk code.

~$ awk '{print $0} END {print "##########\n Completed printing
filtered data\n##########"}' db.txt

Output:

Jones 21 78 84 77
Gondrol 23 56 58 45
RinRao 25 21 38 37
Edwin 25 87 97 95
Dayan 24 55 30 47
##########
Completed printing filtered data
##########

Combining BEGIN and END block

Example:

~$ awk 'BEGIN{print "############################\n This is the output of filtered
data\n############################"}{print $0}END{print
"#########################\n Completed printing filtered
data\n#########################"}' db.txt

Output:

```
#########################
This is the output of filtered data
#########################
Jones 21 78 84 77
Gondrol 23 56 58 45
RinRao 25 21 38 37
Edwin 25 87 97 95
Dayan 24 55 30 47
#########################
Completed printing filtered data
```

awk inbuilt variables

awk is supplied with good number of built-in variables which comes in handy when working with data files. We will see usages of awk built-in variables with one or two examples . These variable are used to format the output of an awk command.

List of built-in variables:

FS field separator character (default blank & tab)
OFS output field separator string (default blank)
RS input record separator character (default newline)
ORS output record separator string (default newline)
NF number of fields in input record
NR number of input record
FNR output number of lines
FILENAME name of current input file

Consider below db.txt as sample file.
~$ cat db.txt
John,29,MS,IBM,M,Married
Barbi,45,MD,JHH,F,Single
Mitch,33,BS,BofA,M,Single
Tim,39,Phd,DELL,M,Married
Lisa,22,BS,SmartDrive,F,Married

In order to make it simple we can divide above inbuilt variables in to groups on basis of their operations.
Group1: FS(input field separator), OFS(Output Field Separator)
Group2: RS(Row separator) and ORS(Output Record Separator)
Group3: NR, NF and FNR
Group4: FILENAME variable

FS (Input Field Separator)

This variable is useful in storing the input field separator. By default AWK can understand only spaces, tabs as input and output separators. But if your file contains some other character as separator other than these mention one's, awk cannot understand them.

For example UNIX password file which contain ':' as a separator. So in order to mention the input filed separator we use this inbuilt variable. We will see what issue we face if we don't mention the field separator for our db.txt.

Example: without using FS

Print first column data from db.txt file.
~$ awk '{print $1}' db.txt

Output:
John,29,MS,IBM,M,Married
Barbi,45,MD,JHH,F,Single
Mitch,33,BS,BofA,M,Single
Tim,39,Phd,DELL,M,Married
Lisa,22,BS,SmartDrive,F,Married

OFS (Output Field Separator)
This variable is useful for defining the output field separator for the expected output data.

Example:

Display only 1st and 4th column and with $ as field separator for the output .
~$ awk 'BEGIN{FS=",";OFS=" $ "}{print $1,$4}' db.txt

Output:
John $ IBM
Barbi $ JHH
Mitch $ BofA

Tim $ DELL
Lisa $ SmartDrive
Note: Space is give before and after $ in OFS variable to show better output.

RS (Row separator)
Row Separator is helpful in defining separator between rows in a file. By default awk takes row separator as new line. We can change this by using RS built-in variable.

Example:
Convert a sentence to a word per line. We can use RS variable for doing it.
~$ echo “This is how it works” | awk ‘BEGIN{RS=” ”}{print $0}’

Output:
This
is
how
it
Works

ORS (Output Record Separator)
This variable is useful for defining the record separator for the awk command output. By default ORS is set to new line.

Example:
Print all the company names in single line which are in 4th column.
~$ awk -F’,’ ‘BEGIN{ORS=” “}{print $4}’ db.txt

Output:
IBM JHH BofA DELL SmartDrive

NF
This variable keeps information about total fields in a given row. The final value of a row can be represented with $NF.

Example: Consider abc.txt which contains below data:
Jones 2143 78 84 77
Gondrol 2321 56 58 45
RinRao 2122234 38 37
Edwin 253734 87 97 95
Dayan 24155 30 47
Print number of fields in each row in abc.txt.
~$ awk ‘{print NF}’ abc.txt

Output:
5
5
4
5
4

NR

This variable keeps the value of present line number. This will come handy when you want to print line numbers in a file.

Example:
Print line number for each line in a given file.
~$ awk '{print NR, $0}' abc.txt

Output:
1 Jones 2143 78 84 77
2 Gondrol 2321 56 58 45
3 RinRao 2122234 38 37
4 Edwin 253734 87 97 95
5 Dayan 24155 30 47
This can be treated as cat command -n option for displaying line number for a file

FNR
This variable keeps count of number of lines present in a given file/data. This will come handy when
you want to print no of line present in a given file. This command is equivalent to wc -l command.

Example:
Print total number of lines in a given file.
~$ awk 'END{print FNR}' abc.txt

Output:
5
FILENAME
This variable contain file awk command is processing.

Example:
Print filename for each line in a given file.
~$ awk '{print FILENAME, NR, $0}' abc.txt

Output:
abc.txt 1 Jones 2143 78 84 77
abc.txt 2 Gondrol 2321 56 58 45
abc.txt 3 RinRao 2122234 38 37
abc.txt 4 Edwin 253734 87 97 95
abc.txt 5 Dayan 24155 30 47

awk Built in Function

A function is a self-contained computation that accepts a number of arguments as input and returns some value. awk has a number of built-in functions in two groups: arithmetic and string functions.

Arithmetic Functions
Nine of the built-in functions can be classified as arithmetic functions. Most of them take a numeric argument and return a numeric value. Below table summarizes these arithmetic functions with some Examples.

awk Function Description
cos (x) Returns cosine of x (x is in radians).
exp (x) Returns e to the power x.
index (s1,s2) Position of string s2 in s1; returns 0 if not present
int (x) Returns truncated value of x.
log (x) Returns natural logarithm (base- e) of x.
sin (x) Returns sine of x (x is in radians)

sqrt (x) Returns square root of x.
atan2 (y , x) Returns arctangent of y / x in the range - to .
rand () Returns pseudo-random number r, where 0 <= r < 1.
sqrt(expr) Returns the square root of the expression or value given

Examples:
```
~$ awk 'BEGIN{
print sqrt(16);
print sqrt(0);
print sqrt(-12);
```

}'

Output:

4

0

nan

Here nan stands for not a valid number.

String Functions

The built-in string functions are much more significant and interesting than the numeric functions. Because awk is essentially designed as a string-processing language, a lot of its power derives from these functions. Below table lists the string functions found in awk. awk's Built-In String Functions

Awk Function	Description
`gsub` (*r* , *s* , *t*)	Globally substitute's *s* for each match of the regular expression *r* in the string *t*. Returns the number of substitutions. If *t* is not supplied, defaults to **$0**.
`index` (*s* , *t*)	Returns position of substring *t* in string *s* or zero if not present.
`length` (*s*)	Returns length of string *s* or length of **$0** if no string is supplied.
`match` (*s* , *r*)	Returns either the position in *s* where the regular expression *r* begins or 0 if no occurrences are found. Sets the values of **RSTART** and **RLENGTH**.
`split` (*s* , *a* , *sep*)	Parses string *s* into elements of array *a* using field separator *sep* ; returns

Awk Function	Description
	number of elements. If *sep* is not supplied, **FS** is used. Array splitting works the same way as field splitting. Uses `printf` format specification for **expr**.
`sprintf` ("*fmt* ", *expr*)	
`sub` (*r* , *s* , *t*)	Substitutes *s* for first match of the regular expression *r* in the string *t*. Returns 1 if successful; 0 otherwise. If *t* is not supplied, defaults to **$0**.
`substr` (*s* , *p* , *n*)	Returns substring of string *s* at beginning position *p* up to a maximum length of *n*. If *n* is not supplied, the rest of the string from *p* is used.
`tolower` (*s*)	Translates all uppercase characters in string *s* to lowercase and returns the new string.
`toupper` (*s*)	Translates all lowercase characters in string *s* to uppercase and returns the new string.